

Bloque 13 - Herramientas y reproducibilidad

Propósito

Un análisis útil no termina cuando aparece una cifra: termina cuando otra persona puede entender qué se decidió, repetir el cálculo, revisar sus límites y usar el resultado sin romperlo. En este bloque Leo trabaja como analista de **Nébula**, una app B2B de gestión de reservas. Tras una versión nueva, la activación cae y Producto debe decidir si corregir, revertir o mantener el cambio.

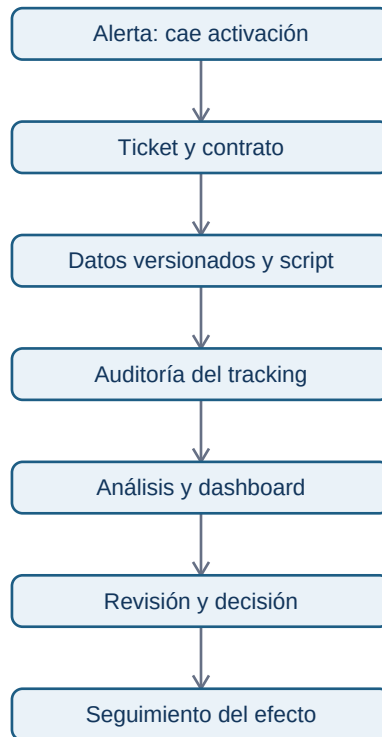
La pregunta continua del bloque es: **¿ha reducido la versión 4.2 la activación de cuentas nuevas y qué evidencia reproducible justifica la siguiente acción?**. El caso conecta ticket, datos, código, instrumentación, dashboard, revisión y seguimiento.

Resultados observables

Al terminar podrás convertir una petición vaga en un contrato de análisis, organizar un proyecto, usar Git sin confundir historia con copia de seguridad, auditar un tracking plan, entregar un dashboard gobernado y documentar una decisión trazable.

Prerrequisitos: Bloques 00-10. No se presupone experiencia con Jira, Git, Amplitude ni una herramienta de BI: se presentan como respuestas a problemas de colaboración concretos.

Mapa del caso



La cadena no afirma que una herramienta garantice calidad. Cada flecha es una evidencia que permite comprobar la siguiente: un panel no puede arreglar una definición ambigua, ni un commit puede justificar una recomendación sin datos válidos.

Lecciones

1. De petición a ticket analítico
2. Proyecto reproducible y Git
3. Notebooks, scripts y revisión de pares
4. Instrumentación, tracking plan y Amplitude
5. BI, dashboards y contrato de métrica
6. Entrega, seguimiento y comunicación

Práctica y laboratorio

Resuelve la [investigación reproducible de activación](#) antes de consultar la [solución razonada](#). Ejecuta el [laboratorio](#) desde el móvil con un intérprete Python online o en tu ordenador; no requiere paquetes externos ni datos personales.

Criterio de dominio

No basta con saber nombrar Jira, Amplitude, Git o Power BI. Debes poder explicar qué evidencia deja cada uno, qué no demuestra, cómo detectar una definición rota y cómo otra persona reproduce tu conclusión.

1. De petición a ticket analítico

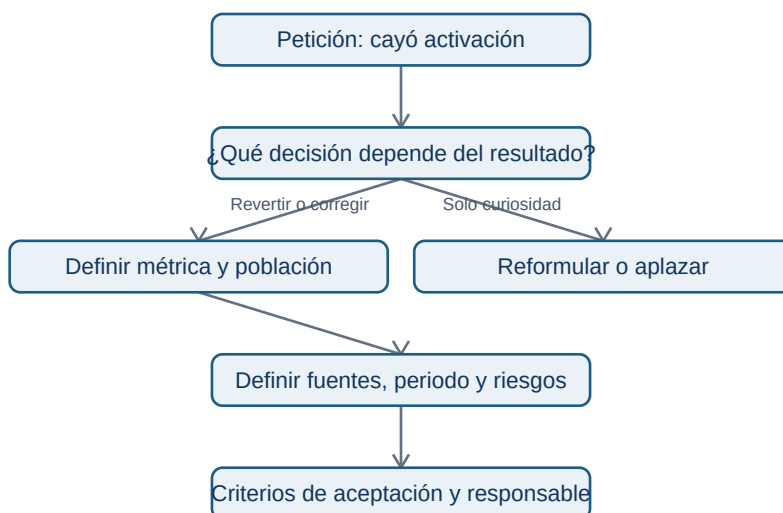
Objetivo y prerequisites

Convertirás «mira por qué baja el onboarding» en un acuerdo que permita tomar una decisión y evaluar si el trabajo está terminado. Necesitas distinguir una métrica y un segmento (bloque 10); no necesitas conocer Jira.

Del mensaje ambiguo al problema decidable

Un equipo recibe: «la activación ha caído; miradlo». Es una **petición**, no una pregunta analítica. Puede significar que cambió el producto, que el evento dejó de llegar, que entró tráfico distinto o que cambió la definición. Si el analista empieza por abrir un gráfico, el resultado puede ser interesante pero inútil.

Un *ticket* es una ficha compartida de trabajo. Jira es una aplicación popular para guardar estas fichas; la idea es independiente de la marca. Su función no es vigilar personas: conserva el contexto, las decisiones y los criterios de aceptación cuando la conversación ya no está en Slack o en la memoria de alguien.



La pregunta del diagrama obliga a fijar el uso antes de medir. «¿Cayó?» no basta; en Nébula se decide entre revertir la versión 4.2, corregir el flujo o mantenerla.

El contrato mínimo de Nébula

- **Decisión:** el PM decide revertir, lanzar corrección o mantener 4.2 el martes.
- **Pregunta:** ¿la tasa de activación a siete días difiere por versión de app tras el lanzamiento?
- **Métrica:** cuentas nuevas que completan reserva_creada en sus primeros siete días / cuentas nuevas elegibles.
- **Población y grano:** una fila por cuenta y fecha de alta; se excluyen cuentas internas y pruebas.
- **Periodo:** altas del 1 al 28 de abril; corte de datos el 6 de mayo para observar siete días.
- **Segmentos:** versión, plataforma, país y canal de adquisición.
- **Evidencia:** consulta versionada, extracción fechada, auditoría del evento y tabla de resultados.
- **Dueño:** Ana (Producto) decide; Leo analiza; Marta (Datos) valida instrumentación.

El detalle evita una trampa frecuente: comparar altas muy recientes contra altas antiguas. Las primeras aún no tuvieron siete días para activarse. La **fecha de corte** y la ventana de observación son parte de la definición, no letra pequeña.

Criterios de aceptación y límites

Un criterio de aceptación no es «hacer dashboard». Para este ticket: (1) la fórmula se puede recalcular desde una fuente identificada; (2) la cobertura de reserva_creada se compara entre versiones; (3) se muestra tamaño de cada grupo, tasa y diferencia; (4) se enumeran riesgos de causalidad; (5) la entrega recomienda una acción o explica por qué aún no puede hacerlo.

No redactes «demostrar que 4.2 causó la caída». Una comparación antes/después observa asociación; coincide con campañas, estacionalidad y cambios de tráfico. Para afirmar causalidad se necesitaría un diseño apropiado, como experimento o una estrategia cuasiexperimental (bloque 14).

Error habitual

«Activación» puede ser pulsar un botón, crear una reserva o recibir confirmación del servidor. Elegir la definición después de ver el resultado es **moving the goalposts**: transforma una investigación en una búsqueda de una conclusión deseada.

Resumen y comprobación

Un buen ticket contiene una decisión, pregunta, contrato de métrica, población, corte temporal, fuentes, riesgos, responsable y criterios de aceptación. Pregúntate: ¿otra persona sabría qué hacer si Leo no está disponible? ¿el resultado puede cambiar una acción concreta?

Continúa con [proyecto reproducible y Git](#): el ticket define qué demostrar; el proyecto conserva cómo se demostró.

2. Proyecto reproducible y Git

Objetivo y problema

Organizarás el análisis de Nébula para que una compañera pueda repetirlo semanas después.

Reproducible significa: con una entrada identificada, una versión de código y parámetros explícitos, se obtiene el mismo resultado o se explica por qué no. No significa subir datos personales a internet.

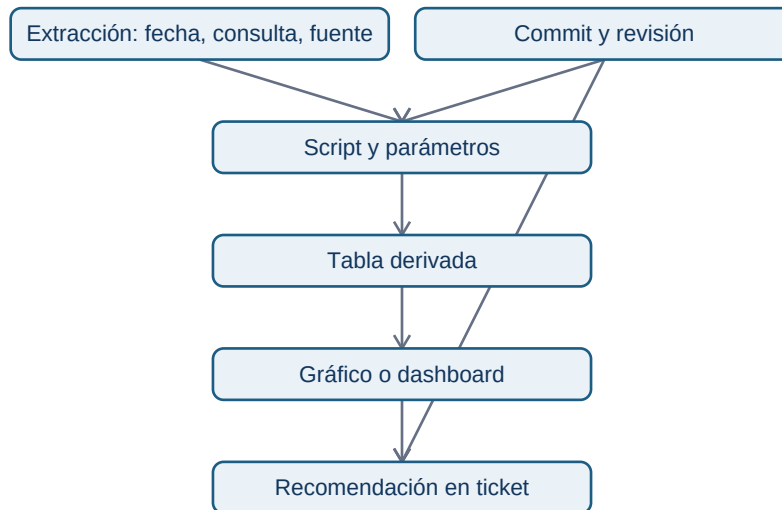
Una **carpeta** agrupa archivos; un **repositorio Git** es una carpeta cuyo historial Git registra cambios en archivos de texto. Git no entiende por sí mismo qué es correcto: guarda quién cambió qué, cuándo y con qué mensaje. La documentación aporta significado.

Estructura mínima que separa responsabilidades

```
activacion-nebula/
■■■ README.md           # propósito, cómo ejecutar y límites
■■■ data/
■   ■■■ raw/            # extracción original; no se versiona si es sensible
■   ■■■ processed/      # derivado reproducible; normalmente tampoco se versiona
■■■ src/
■   ■■■ calcular_activacion.py
■■■ tests/              # comprobaciones de la lógica
■■■ docs/
■   ■■■ ticket.md
■   ■■■ contrato-metrica.md
■   ■■■ tracking-plan.md
■■■ outputs/            # tablas o gráficos regenerables
■■■ requirements.txt     # dependencias y versiones, si existen
```

raw conserva la evidencia original; processed contiene transformaciones que se pueden regenerar; src declara la lógica. Si mezclas copia manual de Excel, resultado final y script sin nombres claros, nadie sabe qué es fuente y qué es consecuencia.

Trazabilidad como cadena de custodia analítica



Para reproducir una conclusión hay que poder recorrer las flechas hacia atrás. Un número en una diapositiva sin consulta, corte ni definición no es auditable aunque sea cierto.

Git en lenguaje de trabajo

Un **commit** es una fotografía etiquetada de una unidad coherente de cambio. Una **rama** permite preparar una propuesta sin alterar la línea principal. Una **pull request (PR)** muestra la diferencia de una rama y abre una conversación de revisión antes de integrarla. GitHub documenta que la revisión permite comentar, aprobar o solicitar cambios; no sustituye ejecutar el análisis ni revisar su significado.

Ejemplo de secuencia:

```
git switch -c fix/definicion-activacion
# editar contrato y script
git status
git add docs/contrato-metrica.md src/calcular_activacion.py
git commit -m "Aclara población elegible de activación"
git push -u origin fix/definicion-activacion
```

El mensaje no dice «cambios varios»: permite entender el propósito sin abrir todos los archivos. Antes de `git add`, `git status` es una pausa de seguridad: evita incluir credenciales, exportaciones sensibles o resultados irrelevantes.

Datos sensibles y entorno

No incluyas identificadores de usuarios, claves de API, contraseñas ni un `data/raw` real en un repositorio. Usa `.gitignore` para prevenir adiciones accidentales, un archivo de ejemplo sintético y un documento que indique quién puede regenerar la extracción y con qué permisos. Una variable de entorno es un valor configurado fuera del código, útil para rutas o secretos; tampoco se imprime en el

informe.

La reproducibilidad total puede fallar si una API cambia, la fuente se actualiza o una librería tiene otra versión. Por ello registra fecha de extracción, versión de dependencias y parámetros. No prometas repetir hoy exactamente una cifra basada en una tabla que cambia cada hora.

Resumen y comprobación

Explica la diferencia entre un commit y una copia de seguridad; entre dato bruto y derivado; entre un archivo ignorado y un archivo inexistente. Después pasa a [notebooks](#), [scripts](#) y [revisión](#): decidirás dónde vive cada parte de la lógica.

Referencia primaria: [GitHub Docs: revisiones de pull request](#).

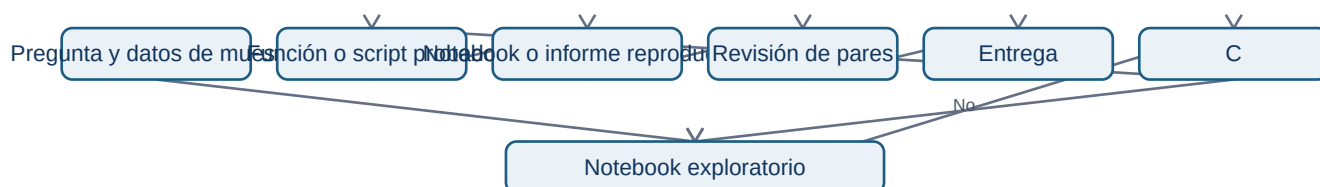
3. Notebooks, scripts y revisión de pares

Objetivo

Elegirás una forma de trabajo que permita explorar sin convertir el resultado en una caja negra. Un **notebook** es un documento con celdas de explicación, código y salida; un **script** es un archivo de instrucciones que se ejecuta de principio a fin. Ambos pueden ser correctos; resuelven problemas distintos.

Exploración, producción y explicación

En Nébula Leo explora una muestra en un notebook: cuenta cuentas por versión, mira nulos y formula hipótesis. Cuando ya sabe la regla de activación, la mueve a `src/calcular_activacion.py` para que se ejecute siempre igual. El notebook final importa esa lógica, explica decisiones y muestra la tabla que revisa Producto.



El bucle de exploración es normal. El peligro aparece si una celda usa un estado oculto: el notebook parece funcionar porque una variable se ejecutó ayer, pero falla desde cero.

Contrato de una función analítica

Una función no es solo sintaxis: declara una entrada, una transformación y una salida esperada.

```
def tasa_activacion(cuentas):  
    """Devuelve activadas / elegibles; exige una fila por cuenta."""  
    elegibles = [fila for fila in cuentas if not fila["es_interna"]]  
    if not elegibles:  
        raise ValueError("No hay cuentas elegibles")  
    activadas = sum(fila["activa_7d"] for fila in elegibles)  
    return activadas / len(elegibles)
```

El comentario expone el **grano** (una fila por cuenta) y el error explícito evita devolver una tasa falsa cuando el denominador es cero. Una prueba pequeña verifica, por ejemplo, que dos activadas de cuatro cuentas dan 0.5 y que las internas no entran en el denominador.

Lista de revisión que importa

Una revisión útil no dice solo «usa otro nombre». Quien revisa debe poder responder:

- ¿la fuente, fecha de corte y consulta están identificadas?
- ¿la unidad de análisis es una cuenta, usuario o evento y se mantiene al unir tablas?
- ¿el denominador coincide con el contrato de métrica?
- ¿hay filtros de pruebas, duplicados, nulos y zonas horarias documentados?
- ¿el notebook se ejecuta de arriba abajo en un entorno limpio?
- ¿la conclusión distingue una caída observada de una causa probada?

Un comentario accionable dice: «En la línea que une eventos con cuentas, valida que cada cuenta siga apareciendo una vez; de lo contrario varios eventos inflan el denominador». Incluye el riesgo y cómo verificarlo.

Contraejemplo

Copiar una tabla desde un notebook al dashboard puede dar una respuesta rápida, pero si no existe script ni parámetros esa tabla no puede actualizarse ni auditarse. Automatizar tampoco es siempre mejor: para una pregunta irrepetible y pequeña, documentar un paso manual controlado puede ser suficiente; lo importante es declararlo.

Resumen

Explora en notebooks, estabiliza reglas en scripts y revisa supuestos, no solo formato. En la siguiente lección comprobarás que un script correcto no arregla eventos que nunca se recogieron:

[instrumentación y tracking plan](#).

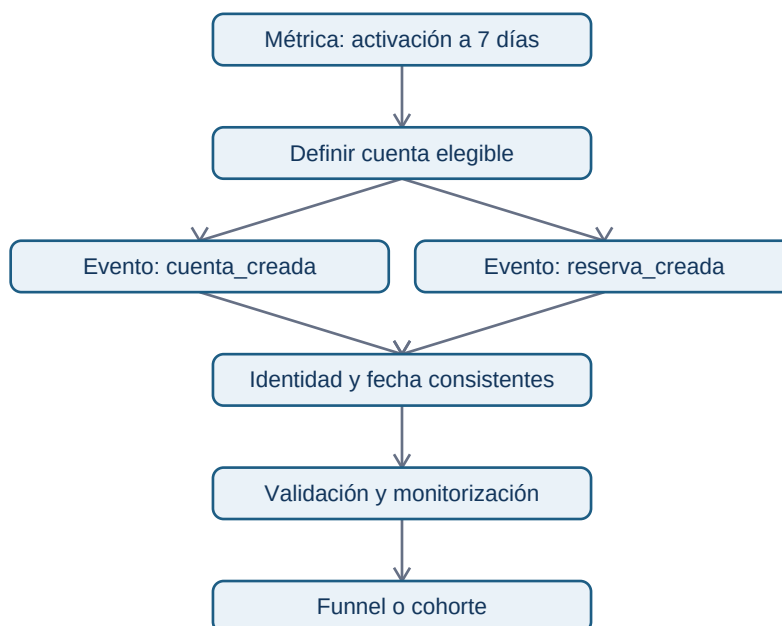
4. Instrumentación, tracking plan y Amplitude

Objetivo y vocabulario

Diseñarás evidencia antes de pedir un funnel. Un **evento** es el registro de que ocurrió una acción en un momento; una **propiedad** describe su contexto. Por ejemplo, `reserva_creada` es un evento y `version_app="4.2"` puede ser una propiedad. Instrumentar es programar el producto para que emita esos registros de forma definida.

Un **tracking plan** es el contrato compartido de esos eventos. Amplitude es una plataforma que puede gestionar ese plan y analizar eventos; no convierte automáticamente datos ambiguos en datos fiables. Su documentación actual recomienda diseñar el plan antes de escalar la instrumentación y define eventos, propiedades y fuentes de emisión.

De una métrica a los eventos necesarios



El diagrama muestra una dependencia: un funnel es el último paso. Si `cuenta_creada` usa un identificador y `reserva_creada` otro, no se puede saber con honestidad si una misma cuenta hizo ambas cosas.

Extracto de tracking plan para Nébula

- **Evento:** cuenta_creada.
- **Cuándo se emite:** el backend confirma la creación; no al abrir el formulario.
- **Identidad:** account_id estable; nunca correo ni nombre en la herramienta analítica.
- **Propiedades:** version_app, plataforma, canal, país, timestamp_utc.
- **Reglas:** version_app es texto no vacío; plataforma está en ios, android, web; fecha en UTC.
- **Dueño:** Ingeniería de plataforma implementa; Producto aprueba la semántica; Datos valida cobertura.
- **Versión:** tracking_plan_v3, fecha de entrada y fecha de retirada.

La propiedad no existe para «capturarlo todo». Recoger país puede ser proporcional para segmentar una versión; recoger contenido de notas de usuario no lo es para medir activación. Minimizar información reduce riesgo de privacidad y complejidad.

Validar antes de interpretar

Antes de comparar 4.1 y 4.2, revisa: volumen diario de cada evento, proporción de propiedades obligatorias presentes, distribución por versión, duplicados y retraso de llegada. Si el evento de reserva dejó de emitirse en Android 4.2, una caída del funnel es un problema de medición, no evidencia sobre comportamiento.

Amplitude Data permite definir eventos, propiedades, fuentes y reglas; también puede señalar datos inesperados o inválidos frente al plan. Esa validación es una ayuda operativa, no una sustitución de la decisión humana sobre qué significa «activación».

Error habitual: cliente frente a servidor

Un evento enviado desde el móvil puede no llegar si la aplicación se cierra o no tiene red. Un evento confirmado por servidor suele representar una acción completada, pero puede llegar con retraso y no refleja abandonos del formulario. El tracking plan debe declarar cuál se usa y por qué; mezclar ambos sin distinguirlos genera doble conteo.

Resumen y fuentes

La instrumentación es un sistema de evidencia con semántica, identidad, reglas, propietarios y versión. Antes de un dashboard, valida que el sistema sigue observando lo que promete.

Fuentes primarias actuales: [crear un tracking plan en Amplitude](#), [planificar la implementación](#) y [monitorizar eventos frente al plan](#).

Sigue con [BI y dashboards](#), donde esa evidencia se convierte en una vista de decisión repetible.

5. BI, dashboards y contrato de métrica

Objetivo

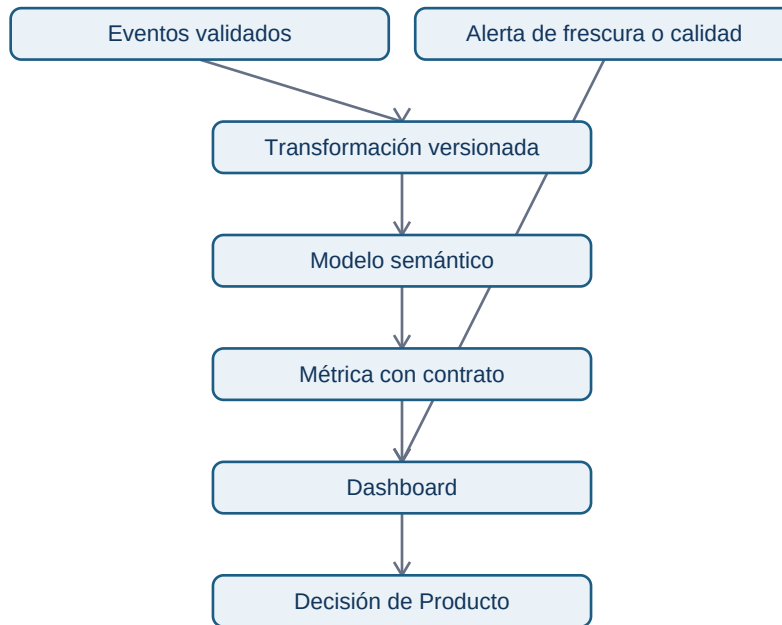
Diseñarás un dashboard que ayude a decidir sin ocultar su definición. Una herramienta de **BI** (business intelligence), como Power BI, Tableau o Looker, conecta datos, modelo semántico, métricas y visuales. La interfaz cambia; el problema permanece: muchas personas necesitan consultar la misma medida sin que cada una escriba una fórmula diferente.

Un dashboard no es una investigación completa ni un mural de gráficos. Es un producto recurrente para una audiencia, una decisión y una cadencia concreta.

El contrato viaja con la métrica

Para Nébula, el título «Activación» no basta. El contrato debe ser visible mediante un enlace o panel de información:

- **Fórmula:** cuentas elegibles con reserva confirmada en siete días / cuentas elegibles dadas de alta.
- **Grano:** una fila por cuenta de alta; no una fila por evento.
- **Ventana:** cohorte por fecha de alta, observada siete días completos.
- **Fuente:** tabla derivada `activation_cohort_v3`, refrescada desde eventos validados.
- **Exclusiones:** cuentas internas, demos y altas sin siete días observables.
- **Frescura:** última actualización, zona horaria y retraso esperado visibles.
- **Propietario:** Datos mantiene la definición; Producto aprueba cambios de propósito.



Cada capa tiene una responsabilidad. Añadir una fórmula rápida directamente al gráfico evita el modelo compartido y multiplica definiciones. La alerta no implica que el dato sea falso, pero evita que una cifra atrasada se lea como presente.

Vista de decisión, no colección de gráficos

El panel de activación puede contener: indicador global y diferencia frente al periodo comparable; tendencia por cohorte; tabla por versión/plataforma con tamaños de muestra; estado de cobertura de los dos eventos; enlace a ticket y contrato. Cada visual responde una pregunta escrita: «¿dónde está la caída?» en lugar de «gráfico 3».

Un filtro puede cambiar el denominador. Si la persona elige Android, el panel debe indicar que analiza solo cuentas Android y preservar la definición temporal. No permitas filtros silenciosos que combinen ventanas distintas o excluyan datos sin aviso.

Refresco, permisos y mantenimiento

El refresco lee de una fuente, actualiza el modelo y actualiza visuales que dependen de él; por eso un dashboard necesita fecha de último refresco y dueño. Power BI documenta estas fases y sus dependencias. Los permisos deben dar acceso al mínimo necesario: no todo consumidor de un panel necesita acceso a identificadores de eventos.

Contraejemplo

Un tablero puede mostrar que Android 4.2 tiene 42 % frente a 48 % en 4.1. No prueba que la versión sea causa: puede haber canales, países o cohortes distintos. El dashboard señala dónde investigar; el ticket, diseño de análisis y límites deciden qué se puede afirmar.

Resumen y comprobación

Un dashboard fiable hace visibles fórmula, grano, frescura, filtros, fuentes y responsable. Comprueba: ¿puedes recrear la métrica fuera del panel? ¿alguien detectaría un fallo de tracking antes de tomar una decisión?

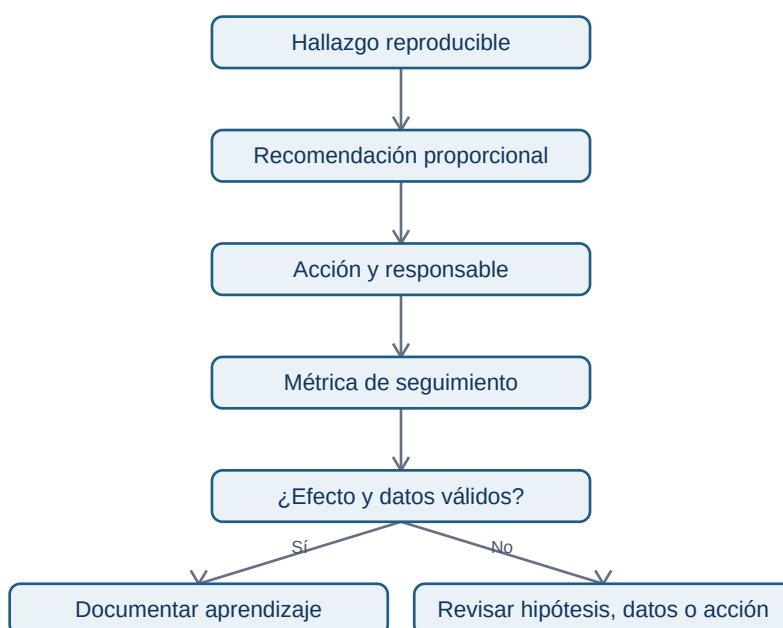
Fuente primaria: [Microsoft Learn: ciclo de actualización de datos en Power BI](#). Continúa con [entrega y seguimiento](#).

6. Entrega, seguimiento y comunicación

Objetivo

Cerrarás un análisis de manera que una decisión sea revisable y genere aprendizaje. La entrega correcta no es «muchos gráficos»: combina una recomendación, la evidencia que la respalda, incertidumbre, artefactos reproducibles y un plan para comprobar el efecto.

La secuencia siguiente muestra qué debe quedar unido cuando el análisis se entrega y se opera:



La secuencia evita dos errores opuestos: defender la primera conclusión por orgullo y cambiar de decisión cada vez que un gráfico se mueve. Si la evidencia no es válida, se abre una nueva investigación; el seguimiento puede invalidar la interpretación inicial y eso es aprendizaje, no fracaso.

La nota de decisión de Nébula

Una nota de una página puede seguir esta estructura:

1. **Decisión solicitada:** no revertir todavía; lanzar corrección de tracking Android y limitar el despliegue de 4.2.
2. **Hallazgo:** la activación observada es menor en Android 4.2, pero `reserva_creada` cae de forma anómala el mismo día de la versión.
3. **Evidencia:** enlace a consulta/script versionado, cohorte, tamaños, cobertura diaria y dashboard.
4. **Qué no sabemos:** no se puede atribuir la caída al flujo de producto hasta validar emisión del evento.
5. **Siguiente medición:** tras corrección, comparar cohorte con siete días completos; responsable y fecha.

Adaptar el formato sin alterar la certeza

Dirección necesita decisión, coste, riesgo y fecha. Ingeniería necesita definición del evento, entorno, criterios de validación y enlace al ticket. Datos necesita consulta, versión de código, corte y controles de calidad. Son vistas del mismo contrato: resumir no autoriza a convertir una asociación en causalidad.

Registra también decisiones negativas: «no se publica la tasa por país porque falta cobertura en Android». Sin ese registro, el equipo puede volver a hacer el mismo análisis defectuoso dentro de tres meses.

Cierre operativo

Antes de cerrar el ticket, confirma que el entregable tiene: enlace al repositorio o script, versión de definición, fuente y fecha de corte, revisión realizada, dueño de la acción, métrica de seguimiento y fecha de revisión. Si se modifica la métrica, abre un cambio nuevo: reescribir el pasado sin versión rompe la comparabilidad.

Límite profesional

Reproducible no equivale a útil. Puedes repetir un cálculo impecable basado en una pregunta irrelevante. Por eso el ticket vuelve a aparecer al final: el análisis sirve a una decisión explícita y sus consecuencias deben observarse.

Práctica

Resuelve la [investigación reproducible de activación](#), ejecuta el [laboratorio](#) y compara tus decisiones con la [solución](#).

Al terminar, el bloque 14 amplía tus herramientas técnicas, pero el hábito de contratos, trazabilidad y seguimiento debe permanecer en todos los proyectos.